RestWebApp - Codebase Overview

Overview

RestWebApp is a Spring Boot 3.2 web application for managing restaurant sales and transactions. It provides a dashboard where authenticated users can track cash, visa, and expense amounts per restaurant, with a login gate powered by Spring Security.

Key Technologies

Layer	Technology
Backend framework	Spring Boot 3.2 (Java 17)
Web	Spring MVC (REST + Thymeleaf)
Persistence	Spring Data JPA + Hibernate
Database	PostgreSQL (port 5432, db: restaurantdb)
Security	Spring Security (form login, in-memory user)
Frontend	Vanilla JS + Chart.js (CDN)
Build	Maven

Directory Structure

`RestaurantApplication.java`	@SpringBootApplication entry point
`controller/HomeController.java`	GET / renders index.html
`controller/LoginController.java`	GET /login renders login.html
`controller/RestaurantController.java`	REST: GET /api/restaurants
`controller/SaleController.java`	REST: GET / POST / PUT / DELETE /api/sales
`model/Restaurant.java`	JPA entity: id, name
`model/Sale.java`	JPA entity: id, restaurant (FK), cashAmount, visaAmount, storeName, spent, totalAm
`repository/RestaurantRepository.java`	JpaRepository<Restaurant, Long>
`repository/SaleRepository.java`	JpaRepository<Sale, Long> + findByRestaurant()
`security/SecurityConfig.java`	In-memory user: demo / demo, CSRF disabled
`resources/application.properties`	DB config, port 8081, ddl-auto=update
`templates/index.html`	Dashboard (Thymeleaf shell, JS-driven)
`templates/login.html`	Login form
`static/css/theme.css`	Light / dark theme styles
`static/js/app.js`	All frontend logic

How It Works

1. Authentication

Spring Security enforces login for all routes except /login, /css/**, and /js/**.
Credentials are hardcoded in memory as demo / demo.

2. Data Model

A Restaurant has many Sales. Each Sale stores cashAmount, visaAmount, and spent,
and auto-calculates: totalAmount = cash + visa - spent via @PrePersist / @PreUpdate.

3. REST API (/api/...)

GET /api/restaurants - lists all restaurants
GET /api/sales?restaurantId=X - lists sales for a restaurant
POST /api/sales - creates a new sale
PUT /api/sales/{id} - updates a sale
DELETE /api/sales/{id} - deletes a sale

4. Frontend (app.js)

Fully JS-driven SPA-style dashboard. On load it fetches restaurants, populates a dropdown, then fetches sales for the selected restaurant. Features:

- Add / edit / delete transactions via the REST API
- Client-side search, column sorting, and pagination (10 rows / page)
- Doughnut chart (Chart.js) showing cash / visa / expense breakdown
- Light / dark theme toggle persisted in localStorage

5. Database

Schema is auto-managed by Hibernate (ddl-auto=update) against a local PostgreSQL instance on port 5432, database: restaurantdb.

Notable Observations

- No tests exist in the repository.
- Security uses {noop} password encoding (plaintext) -- acceptable for demos, not for production.
- CSRF protection is disabled.
- The Sale entity has a mapping inconsistency: storeName is declared as both a public field and a @Column-annotated getter/setter, which can cause JPA confusion.
- The app runs on port 8081 (not the default 8080).